Python 程序设计课程报告

—— 地下城探险 Roguelike 游戏的设计与实现

课程名称： Python 程序设计

实验名称： 地下城探险 Roguelike 游戏

学生姓名：

学　　号：

专　　业：

指导教师：

2026 年 11 月

注：姓名、学号、专业、指导教师四项为占位，请按实际信息填写。

目　　录

实验目的.....1
实验内容与任务描述.....2
1. 项目概述.....2
2. 核心玩法.....2
3. 功能清单与评分点对应.....3
实验环境.....4
需求分析与设计思路.....5
1. 程序流程图（必选）.....5
2. 模块功能设计.....7
3. 数据结构设计.....9
4. 战斗系统的策略化设计.....10
5. 难度曲线与平衡设计.....12
关键伪代码展示与说明.....15
1. 地牢生成.....15
2. 视野计算.....16
3. 战斗结算.....16
4. 难度调度.....17
5. 存档读写与随机数恢复.....18
实验结果与界面展示.....19
1. 开始界面与设置.....19
2. 主界面与探索.....20
3. 策略战斗.....22
4. 弹窗与交互.....22
5. 数据可视化.....25
6. 数值平衡的迭代过程.....29
7. 界面性能优化.....30
8. 可替换的图标资源.....30
测试与异常处理.....32
1. 测试方法.....32
2. 测试覆盖的重点.....32
3. 三组重点测试记录.....33
4. 异常处理策略.....35
实验总结.....36
1. 完成情况.....36
2. 主要收获.....36
3. 存在的不足与改进方向.....37
4. 结语.....37
参考文献.....38

实验目的

本次 Python 程序设计课程大作业以“做出一个完整可运行的软件作品”为目标，选择**地下城探险 Roguelike（回合制地牢探险游戏）**作为题目。Roguelike 这一类型的特别之处在于：地图、怪物、道具都要程序化生成，玩家每一局的体验都不同，因此它天然要求程序具备“用规则生成内容”的能力，而不是把内容一条条写死。这正好能把课程中的语法、函数、面向对象、文件读写、异常处理与第三方库串成一条完整的工程链路。具体希望达成四个目标。

第一，把分散的语法知识组织成可维护的工程结构。 课程中分别学过变量与流程控制、函数与模块、类与继承、JSON 读写、异常捕获，但单点练习很难体会它们如何配合。本次作业要求从零构建一个具备地图生成、回合制战斗、道具与装备、存档读档、数据统计与图形界面的完整程序，迫使自己回答“这段逻辑应该放在哪个模块里”这类只有真实项目才会遇到的问题。

第二，掌握“逻辑与界面分离”的工程方法。 很多同类作业把业务规则写在按钮回调里，结果是既没法自动化测试、也很难改。本次有意把项目切成 `dungeon/`（纯逻辑层）与 `ui/`（表现层）两个包，并把“逻辑层不得导入界面库”这条约定写成可执行的测试。这样做的收益是具体的：153 个单元测试可以在没有显示器的环境下跑完，批量模拟器也能在命令行里跑几十局对局。

第三，学会用数据而不仅是手感来验证设计。 难度是否合理、一局要打多久、玩家最常死在哪一层，这些问题靠“自己玩两把”回答不了。本次实现了一个无界面的批量模拟器，用同一套逻辑层跑几十局并生成图表，把数值平衡从“拍脑袋”变成“看数据”。报告中的难度曲线正是在这个工具的支持下迭代出来的。

第四，养成规范与文档意识。 代码统一使用类型注解与中文注释，模块职责写进文档字符串；报告要讲清设计取舍、画出流程图、给出可复现的测试证据。这些是后续课程设计与毕业设计的基本功。

实验内容与任务描述

1. 项目概述

本项目名为**地下城探险（Dungeon Delve）**，是一个运行在 Windows 桌面上的回合制 Roguelike 游戏。玩家扮演一名冒险者，从地下城第 1 层出发逐层向下探索，最终在第 10 层击败 BOSS“深渊领主”完成通关。每局的地图、怪物、道具与房间都由随机种子程序化生成，同一颗种子可以完整复现整局。

程序完全使用 Python 实现：界面基于标准库 `tkinter`，数据统计使用 `matplotlib` 与 `pandas`，**不依赖任何外部图片素材**——地图上的墙、地面、门、楼梯、角色与怪物全部由画布图元现场绘制（也支持替换为 PNG 图标，见后文）。因此把项目复制到任何一台装有 Python 的电脑上都能直接运行。

2. 核心玩法

游戏的核心是“每回合只做一个有代价的选择”。玩家每回合只能在移动、攻击、等待之间选一个，而攻击需要先装备武器、再用鼠标选定目标区域；敌人出手前会锁定一片打击区域并显示倒计时，玩家可以走出该区域躲掉这一击，但走位的那一回合就打不出输出。这条张力贯穿始终。主要玩法要素见表 1。

表 1　核心玩法要素

要素	设计说明
地图	40 × 25 格网格，由 7~12 个随机房间与 L 型走廊连通，每格 24 像素
探索	半径 7 格视野，Bresenham 射线判断遮挡；已探索但不可见的区域压暗，未去过的全黑
战斗	回合制策略：每回合一个行动；攻击需装备武器并选定区域；蓝色为可打击范围，彩色为敌人已锁定的区域
成长	击杀获得经验，升级提升生命上限、攻击与防御并回满生命
装备	近战 / 远程 / 护甲三个槽位；同名装备自动合成升级（最高 +3）；只允许更强的装备替换
道具	药水、武器、护甲、卷轴四类共 14 种
敌人	11 种，含远程、直线穿刺、横扫、治疗兵与 BOSS；另有“精英”强化个体

要素	设计说明
房间	四类趣味房间：治疗泉、宝库、陷阱房、力量祭坛
难度	锯齿型逐层剖面 + 三档难度预设 + 可选的自适应难度
存档	3 个槽位，JSON 格式，支持原子写入、版本迁移与损坏自愈
统计	每局记录 14 项指标，可生成 5 张 matplotlib 图表

3. 功能清单与评分点对应

把作业要求逐条落到具体实现上，既是为了自检，也方便评阅时按图索骥。对应关系见表 2。

表 2　功能清单与评分点对应

评分项	本项目对应的实现
输入	键盘（方向键 / WASD / 功能键）与鼠标（点击移动、悬停查看、右键瞄准）两套操作；输入框与列表控件做焦点隔离
逻辑判断	碰撞检测、视野遮挡、攻击形状命中判定、敌人蓄力状态机、命中/暴击/伤害结算、装备比较、胜负判定
文件读写	JSON 存档的保存与读取、历史战绩追加、损坏存档备份、用户设置持久化
统计与可视化	每局 14 项指标落盘；pandas 聚合成 5 张图表并嵌进 tkinter 窗口
扩展功能	战争迷雾、精英怪、治疗兵、四类趣味房间、锯齿难度曲线、自适应难度、攻击范围预览、敌人蓄力预告、伤害飘字、排行榜、批量模拟器、可替换图标
异常处理	坏 JSON、版本过新、缺字段、越界坐标、空背包、非法索引、背包满、配置字段类型错误
界面友好	开始界面、设置面板、全屏、操作说明、日志分色、状态栏提示、悬停查看详情

实验环境

开发与测试均在下列环境中完成。程序只依赖标准库即可运行，第三方库仅用于统计图表与自动截图，缺少它们时游戏本体仍可正常启动。

表 3　实验环境

项目	配置
操作系统	Windows 11（2560 × 1600，150% 缩放）
Python 版本	Python 3.14.2（代码同时兼容 3.10 及以上）
图形库	tkinter 8.6（Python 标准库自带）
可视化库	matplotlib 3.11.2、pandas 3.1.0
辅助库	Pillow 13.0.0（自动截图与图标缩放）
开发工具	Visual Studio Code
运行方式	python main.py，或双击 run.bat

需要说明的是，开发过程中发现部分精简版 Python 发行包**不包含 tkinter**，会导致程序无法启动。为此程序入口 `main.py` 加入了环境自检：缺少 tkinter 时给出可读的中文提示与解决办法，而不是抛出一段难以理解的 `ModuleNotFoundError` 堆栈。

需求分析与设计思路

1. 程序流程图（必选）

游戏主循环采用“结算蓄力 → 玩家行动 → 怪物行动 → 重算视野 → 判定状态”的回合制结构，全部在单线程内同步完成。**不使用多线程**是刻意的选择：tkinter 的控件只能在创建它的线程里访问，一旦把战斗放到后台线程，就要处理跨线程刷新与竞态，得不偿失。主循环流程见图 1。

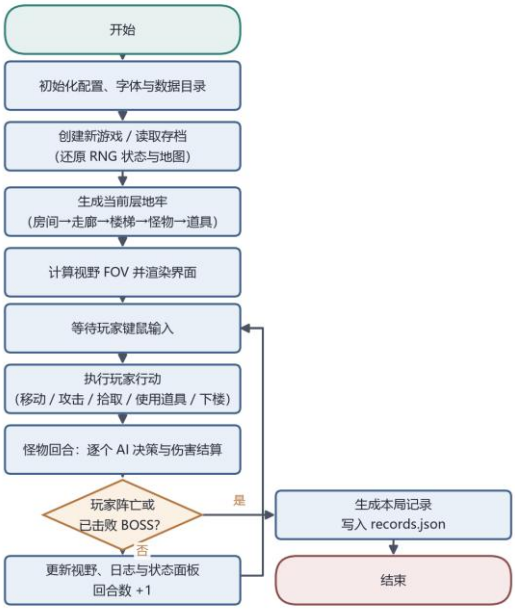


图 1　游戏主循环流程图

地图生成采用“随机房间 +L 型走廊”方案而不是 BSP 分割：核心逻辑只有二十来行，房间连通性天然成立，也便于用伪代码描述。生成流程见图 2——两个“重试”分支分别对应“房间重叠”与“房间数量不够”两种失败情况。

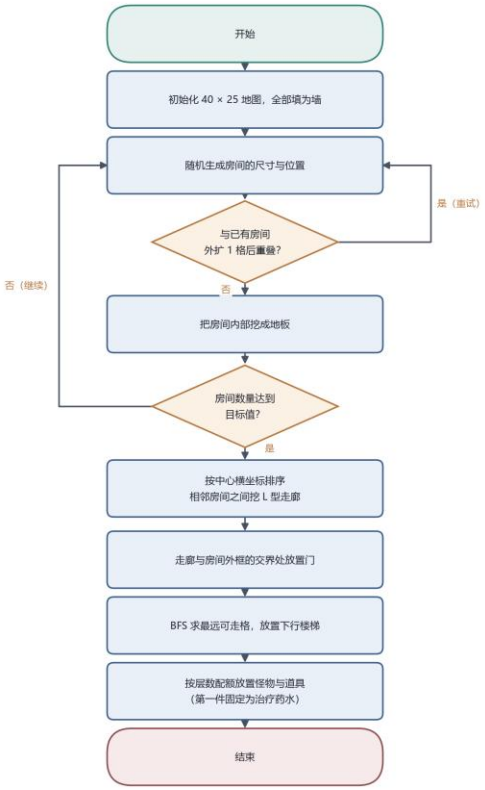


图 2　地牢地图生成流程图

战斗结算被抽成纯函数，不修改任何对象状态，因此可以用固定随机序列写出完全确定的单元测试。“是否命中 → 扣血 → 是否死亡 → 是否通关”这条链路见图 3。

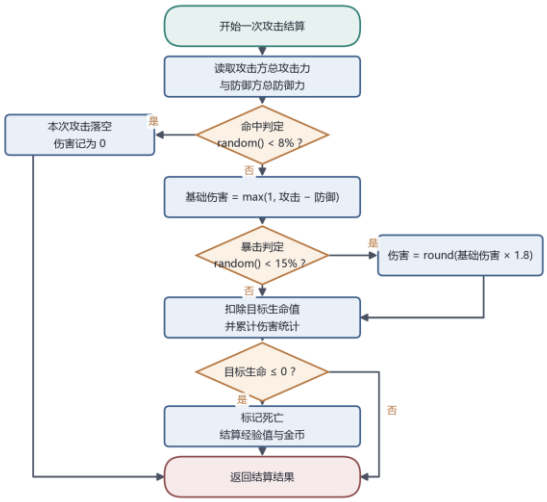


图 3　战斗结算流程图

2. 模块功能设计

项目共 25 个核心源文件、6913 行代码（不含测试与工具脚本），按职责分为逻辑层与表现层两包。逻辑层的每个模块都可以脱离界面单独运行，表现层则不含任何业务规则。各模块职责见表 4。

表 4　模块分层与职责

层次	模块	职责
逻辑层	config.py	集中存放全部常量：地图尺寸、数值平衡、难度剖面、界面配色参数
逻辑层	rng.py	封装 random.Random，提供种子与内部状态的导出/恢复
逻辑层	dungeon.py	地形枚举、房间、房间玩法标记、地图生成、视野、BFS、序列化
逻辑层	entity.py	实体基类：坐标、生命、受伤、治疗、基础序列化
逻辑层	player.py	玩家：等级经验、背包、三个装备槽、强化合成、本局统计
逻辑层	monster.py	怪物物种表（含治疗兵）、精英强化、三种 AI
逻辑层	item.py	道具基类与四类子类、掉落表、换装门槛、工厂与反序列化
逻辑层	patterns.py	攻击形状：近战 / 直线 / 横扫 / 远程，纯函数可单测

层次	模块	职责
逻辑层	combat.py	纯函数战斗结算：命中、暴击、伤害公式
逻辑层	game_state.py	回合状态机、难度调度、房间触发、事件队列、胜负判定
逻辑层	save_manager.py	存档读写、版本迁移、原子写入、战绩追加
逻辑层	stats.py	对局数据聚合（纯 Python 指标 + 可选 pandas 转换）
表现层	theme.py	配色表、字体探测、DPI 感知、ttk 样式
表现层	icons.py	图标清单解析、校验与三级键查找（纯数据）
表现层	icon_set.py	图标加载、缩放、迷雾变暗版生成与缓存
表现层	map_view.py	Canvas 地图渲染：图元缓存、增量刷新、迷雾、飘字、威胁区
表现层	panels.py	状态面板（血条/属性/装备/背包）与日志面板
表现层	dialogs.py	帮助、背包、结算、排行榜、图表、设置六类窗口
表现层	settings.py	设置的 schema、校验与持久化（纯数据，可无头单测）
表现层	start_screen.py	开始界面覆盖层
表现层	charts.py	用 matplotlib 构建 5 张统计图表
表现层	app.py	主窗口装配、菜单、键鼠绑定、设置应用、回合调度

分层的关键约定是：逻辑层不得导入任何界面库。 这条约定没有停留在文档里，而是写成了可执行的测试 `test_architecture.py`——它扫描 `dungeon/` 下所有源文件，一旦出现 `import tkinter` 或 `import matplotlib` 就判定失败。收益非常直接：

- 153 个单元测试在无显示器环境下约 0.35 秒跑完，随时可以回归；
- 批量模拟器可以在命令行里跑几十局，用于验证数值平衡；
- 将来若把界面换成网页或终端，业务代码一行都不用改。

界面层同样不含业务规则：它只调用 `GameState` 的五个意图方法（`try_move`、`wait`、`interact`、`use_item`、`descend`），再从 `state.events` 事件队列单向消费表现事件（例如“在坐标 (12, 7) 弹出 −6 的红字”）。界面层甚至不知道伤害是怎么算出来的。工程规模见表 5。

表 5　工程规模

组成部分	文件数	行数
逻辑层 dungeon/	13	3439
表现层 ui/	11	3405
入口 main.py	1	69
测试 tests/	11	1992
辅助脚本 tools/	10	3196
合计	—	12101

3. 数据结构设计

（1）实体继承体系。 `Entity` 是抽象基类，只负责“有坐标、有生命、能受伤”这些最基础的能力；`Player` 与 `Monster` 继承它并各自扩展。把 `take_damage` 与 `heal` 上提到基类，代码只写一次、两者共用，这是降低重复率的直接手段。

```
Entity(ABC)                    // 坐标 / 生命 / 受伤 / 治疗
 ├── Player                     // 等级、经验、金币、背包、三装备槽、强化合成
 └── Monster                    // 物种、精英标记、AI 类型、蓄力状态、治疗量
```

道具使用独立的继承体系而不继承 `Entity`，因为道具不占据地图实体位，而是挂在地图的坐标字典上；这样“地面上一格只能有一件道具”由字典的键天然保证。

```
Item(ABC)                      // key / name / desc / title（带强化等级）
 ├── Potion    heal_amount
 ├── Weapon    atk_bonus / slot / pattern / reach / charge / upgrade_level
 ├── Armor     def_bonus / upgrade_level
 └── Scroll    effect / power
```

（2）地图表示。 地图用两层的 `list[list[Tile]]` 存储，`Tile` 是 `IntEnum`（墙 0/ 地板 1/ 门 2/ 楼梯 3）。行优先的二维列表便于直接切成字符串存档：25 行 ×40 字符，体积约 1 KB，人眼可读、可 diff，比嵌套的 JSON 二维数组干净得多。视野使用两个集合：`visible`（当前可见）与 `explored`（曾经看到过），界面只读这两个集合决定格子的三种配色，不做任何遮挡计算。

趣味房间不保存房间几何，而是保存**“坐标 → 玩法”的字典**（`features`）。原因是存档只恢复地形字符、不恢复 `Room` 对象；只有坐标化之后的标记才能在读档后继续生效。已触发的房间另用一个集合 `used_features` 记录，保证治疗泉与祭坛每层只能用一次。

（3）回合状态。 `GameState` 持有随机源、玩家、地图、怪物列表、地面道具字典、楼层、回合数与状态标志，是本项目唯一的“可变状态中心”。它对外只暴露五个意图方法，内部则是一条固定的回合流水线。

```
玩家意图(移动/攻击/等待/使用道具/下楼)
  → 结算玩家蓄力（重武器在回合开始落下）
  → 执行玩家行动
  → 房间机制触发（治疗泉 / 陷阱 / 祭坛）
  → 自动拾取
  → 怪物阶段（逐只：治疗 → 推进蓄力 → 起手 → 走位）
  → 重算视野 → 判定死亡/通关/超时 → 推送表现事件
```

（4）存档结构。 存档是一个自描述的 JSON 对象，顶层字段包括 `version`、`meta`（摘要）、`rng`（种子与状态快照）、`player`、`dungeon`、`monsters`、`items`、`log` 与 `progress`。其中 `cls` 字段承担多态反序列化的分发键——读档时先用 `cls` 决定实例化哪个类，再按 `key` 从数据表重建数值，因此即使有人手工把存档里的 `heal_amount` 改成 9999，也无法造出作弊药水。

4. 战斗系统的策略化设计

初版战斗是“撞上去就打”，玩家一回合内既能移动又能攻击，敌人的伤害也是瞬间结算的。这样的战斗没有取舍，也谈不上策略。改造后的目标是：**每个回合都必须做一次有代价的选择**。

（1）每回合只有一个行动。 移动一格、发动一次攻击、原地等待，三者互斥。这条规则让“往前走一步”本身变成决策——走进去可能正好踩在敌人锁定的区域里。

（2）攻击必须先装备武器，再用鼠标选定区域。 武器不再只是加攻击力的数字，它带着完整的攻击描述：形状、射程、蓄力回合。四种形状见表 6。

表 6　武器攻击形状

形状	覆盖范围	代表武器
近战 melee	正前方 1 格	铁剑 +2、符文长剑 +5、深渊之刃 +7
穿刺 line	正前方一条直线，遇墙即止	骨矛 +3（3 格）
横扫 arc	正前方 1 格 + 左右斜前 + 正前方第 2 格	巨刃 +9（需蓄力 1 回合）
远程 ranged	点哪打哪，受最小/最大射程与视线限制	短弓 +3（2~4 格）、长弓 +6（2~6 格）

鼠标悬停时，这一击会覆盖的格子用**蓝色**实时画出来，点下去才算数。校验不只是“这个方向有没有格子”，还要求**点中的那一格确实落在形状里**——否则拿短剑点远处也会被判合法，实际却砍在空地上。

（3）蓄力与锁定预告。伤害越高的武器或敌人，出手前要花越多回合蓄力。敌方起手时会把即将打击的区域**锁定**并逐只上色，在敌人格子右上角显示剩余回合数；锁定后区域不再跟随玩家移动，所以玩家可以走出去躲掉这一击。多只敌人同时蓄力时，每只敌人一种颜色、每片区域描一圈同色边框，避免“一片红糊在一起分不清是谁的范围”。

（4）兵种设计。 全部兵种遵守“伤害越高、蓄力越长”（同档可并列，由单元测试守住）。基础兵种与新增兵种见表 7。

表 7　敌人兵种

兵种	行为	形状 / 射程	攻击	蓄力
史莱姆	游荡	近战 1 格	3	2
巨鼠	追击	近战 1 格	4	2
哥布林	追击	近战 1 格	5	2
吸血蝙蝠	追击	近战 2 格	6	2
萨满巫医	支援	治疗半径 4 格	2	2
弓兵	追击	远程 5 格	6	2
骷髅兵	追击	近战 1 格	7	2
长矛兵	追击	直线 3 格	9	3
石像鬼	守卫	近战 1 格	9	3

兵种	行为	形状 / 射程	攻击	蓄力
大刀兵	追击	横扫 身前	14	4
深渊领主	追击（BOSS）	横扫 身前	23	4

（5）精英怪。 同兵种的强化个体：生命 ×1.75、攻击 ×1.32、防御 +1、经验 ×2.2、金币 ×2.4，名称加“精英·”前缀，地图上用金色描边区分。刻意的约束是**形态与蓄力回合保持不变**——精英是数值修饰而不是新物种，否则会出现“精英史莱姆蓄力 4 回合”这种与体型不符的手感，也会破坏“伤害越高蓄力越长”这条兵种设计规则。

（6）治疗怪。 萨满巫医不攻击玩家，每回合给**血量比例最低**的同伴回血；没有可治疗对象时会主动后退拉开距离，而不是站桩发呆——这样才会形成“先杀治疗”的真实压力。给自己治疗只能回一半，避免变成打不死的自愈机器。地图上用青绿色描边标出，提示玩家优先击杀。

5. 难度曲线与平衡设计

难度曲线没有沿用“每层 +3 只怪”的线性递增，而是先查文献、再定结构。三条依据如下。

其一，心流与“张弛”。 Csikszentmihalyi 在《Flow: The Psychology of Optimal Experience》（1990）中指出，挑战低于能力会无聊、高于能力会焦虑，只有位于两者之间的窄带才能维持沉浸。Schell 在《The Art of Game Design》第 9 章进一步指出，在通道内**锯齿前进**比直线爬升更有趣——“太多紧张会疲惫，太多放松会无聊”（*too much tension, and we wear out. Too much relaxation, and we grow bored*）。

其二，兴趣曲线的释放节拍。 同一本书提出的兴趣曲线认为，健康的强度曲线是“上升 → 尖峰 → 快速释放 → 更高的下一个尖峰”，且释放的时长要与前面的峰值成比例。这解释了为什么“一路加压到底”的游戏反而让人疲劳。

其三，动态难度必须克制。 Hunicke 在《The Case for Dynamic Difficulty Adjustment in Games》（ACE '05, pp.429–433）中提出，难度调整必须在不破坏

核心体验的前提下进行；Adams 在《The Designer's Notebook》中补充了三条工程约束：只增强对手而不削弱玩家、小步频繁而非一次大变、并且应当可以关闭。

据此把逐层剖面写成一张**有峰有谷的表**（表 8）：尖峰层之后紧跟一个释放层，释放层配恢复类房间。数量序列为 5 → 8 → 12 → 9 → 13 → 16 → 20 → 14 → 20 → BOSS，其中第 4、8 层是明显的回落。

表 8　逐层难度剖面

层	怪物	精英	特色房间	设计意图
1	5	0	治疗泉	钩子：低压力教学
2	8	0	—	建立：引入远程与治疗兵
3	12	1	陷阱房	第一个尖峰
4	9	0	治疗泉	释放：恢复与降温
5	13	0	力量祭坛	攀升
6	16	1	—	加压
7	20	2	宝库	高峰：风险与回报
8	14	0	治疗泉	释放：为决战蓄力
9	20	2	力量祭坛	决战前夜
10	6 + BOSS	0	—	高潮

这张表不是“写完就算”，而是被两条单元测试钉住的：一条检查它**确实不是单调递增**（至少两处回落），另一条检查每个尖峰层的下一层都配了恢复房间。把设计意图写成断言，改坏了测试会红。

四类趣味房间是曲线上的“节拍器”：治疗泉与宝库是释放，陷阱房与祭坛是玩家**主动选择**的尖峰——可以绕开，也可以为了收益承担风险，这正是 Schell 所说的风险回报三角形。具体数值见表 9。

表 9　四类趣味房间

房间	效果	设计约束
治疗泉 ≈	回复 28% 最大生命	每层一次，作为尖峰后的补给点
宝库 $	房间内额外 4 件道具 + 2 名守卫	道具按更深一层抽取，守卫随难度档位变化

房间	效果	设计约束
陷阱房 ×	扣除 12% 当前生命，中心放一件好道具作诱饵	先挨一下再拿到奖励，可自行权衡
力量祭坛 †	献祭 15% 当前生命，永久 +2 攻击	生命低于 30% 时拒绝交易，且不消耗机会

自适应难度按 Hunicke 的约束做成“小步、隐蔽、只加强对手”：玩家血量 ≥80% 且击杀效率高于进度时，下一层多 2 只小怪；血量 ≤35% 时少 2 只；上限 ±2，并且不改变层与层之间的锯齿结构，因此玩家察觉不到“游戏在放水”。它可以在设置里关闭，关闭后完全按表 8 执行。

另有**三档难度预设**（轻松 / 标准 / 硬核），只调整对手的数量、精英比例与生命倍率，不修改玩家已获得的能力——这也是 Adams 强调的一条：提高难度应当加强对手，而不是偷偷削弱玩家。

关键伪代码展示与说明

按照模板要求，本节只展示关键逻辑的伪代码，不直接粘贴实现代码。每段伪代码后面说明一到两个关键取舍。

1. 地牢生成

```
算法 GenerateDungeon(层数 floor, 随机源 rng):
    建立 40 × 25 网格，全部填为 WALL
    target ← min(7 + (floor − 1), 12)      // 房间数上限 12
    attempts ← 0
    while 房间数 < target 且 attempts < 240:
        w ← rng(4, 10);  h ← rng(4, 10)
        x ← rng(1, 宽 − w − 2);  y ← rng(1, 高 − h − 2)
        if 新房间与任一已有房间“外扩 1 格”后相交:
            attempts ← attempts + 1;  continue     // 重叠则重新随机
        把新房间加入房间列表
    for 每个房间 r:  把 r 的内部全部挖成 FLOOR

    按房间中心的横坐标升序排序
    for 相邻的两个房间 (prev, cur):
        CarveCorridor(prev 中心, cur 中心)   // L 型走廊
    for 每个房间的外框格:
        if 该格是 FLOOR:  改为 DOOR              // 走廊与房间交界处形成门

    dist ← BFS(玩家出生点)
    楼梯位置 ← argmax(dist)                    // 放在最远处，逼玩家探索
```

两个取舍值得说明。其一是**链式连接**：把房间按中心横坐标排序后依次连接，只要房间数大于 1，整张地图必然连通，不需要额外做连通性检查（测试用 100 个随机种子验证了这一点）。其二是**楼梯取最远点**：如果楼梯随机放，玩家可能“出门就下一层”，整层地图形同虚设；取 BFS 最远点则强制玩家探索一段距离。

2. 视野计算

```
算法 ComputeFOV(玩家坐标 (px, py), 半径 R = 7):
    清空 visible
    把 (px, py) 加入 visible，并加入 explored
    for dy from −R to R:
        for dx from −R to R:
            if dx² + dy² > R²:  continue              // 只取圆形范围
            (x, y) ← (px + dx, py + dy)
            if 越界 或 (x, y) 就是玩家位置:  continue
            path ← Bresenham((px, py) → (x, y))
            blocked ← false
            for 除终点外的每一格 c in path:
                if c 是墙:  blocked ← true;  break
            if not blocked:
                visible ← visible ∪ {(x, y)}
                explored ← explored ∪ {(x, y)}
```

有一个容易被忽略的细节：判断遮挡时**只检查中间格、不检查终点**。这样墙本身是可见的（否则玩家看不到墙壁轮廓，地图会缺一圈），但墙后面的格子不可见，符合直觉。半径取 7 时每回合约需 150 次连线判断，实测单帧耗时不足 1 毫秒，不影响手感。

3. 战斗结算

```
函数 ResolveAttack(攻击方 atk, 防御方 defense, 随机源 rng):
    if rng.random() < 0.08:                    // 8% 落空
        return { hit: false, damage: 0 }
    damage ← max(1, atk − defense)             // 关键：伤害下限为 1
    crit ← (rng.random() < 0.15)               // 15% 暴击
    if crit:  damage ← round(damage × 1.8)
    return { hit: true, crit: crit, damage: damage }
```

`max(1, ...)` 这个下限是必须的。早期版本没有它，当玩家防御成长超过怪物攻击时，会出现“双方都只能造成 0 点伤害”的死循环。加上下限后最坏情况也只是“打得很慢”。这个函数是**纯函数**：不修改任何对象、不读写全局变量、随机源由参数注入，因此测试可以传入一个“按脚本返回数值”的假随机源，把落空、普通命中、暴击三种情况都写成百分之百确定的断言。

4. 难度调度

```
函数 BuildFloor(层数 floor):
    profile ← FLOOR_PROFILE[floor]                  // 锯齿剖面表
    preset  ← DIFFICULTY_PRESETS[当前难度]           // 轻松/标准/硬核

    怪物数 ← round(profile.怪物数 × preset.怪物倍率)
    精英数 ← round(profile.精英数 × preset.精英倍率)

    // 可选的自适应：只调整对手，幅度上限 ±2
    if 自适应已开启:
        if 玩家血量比例 ≥ 0.80 且 击杀进度超前:  怪物数 ← 怪物数 + 2
        else if 玩家血量比例 ≤ 0.35:            怪物数 ← 怪物数 − 2
    怪物数 ← clamp(怪物数, 2, 24)

    生成地图 → 放楼梯 → 按权重抽兵种并放置怪物
    if 精英数 > 0 且 兵种不是治疗兵:  标记为精英（数值加成）
    放置道具 → 按 profile.房间 指定一处趣味房间并补充内容
```

这里的关键是**自适应只作用在“数量”一个维度上**，而且上限只有 ±2 只。如果连精英比例、怪物强度一起动态调整，玩家很容易感觉到“游戏在针对我”，这正是 Adams 所警告的“完美的实时平衡会毁掉节奏”。把调整限制在最小范围，才能既拉回极端体验、又不破坏已建立的压力结构。

5. 存档读写与随机数恢复

```
函数 SaveGame(游戏状态 state, 槽位 slot):
    数据 ← state.ToDict()
    数据.version ← 3
    数据.rng.seed  ← state.rng.seed
    数据.rng.state ← state.rng.GetState()        // 625 个整数的快照
    文本 ← JSON 序列化(数据, ensure_ascii = false, indent = 2)
    写入 “<slot>.json.tmp” → fsync 落盘
    os.replace(“<slot>.json.tmp”, “<slot>.json”)   // 原子替换

函数 LoadGame(槽位 slot):
    if 文件不存在:  抛出 SaveError(“该槽位是空的”)
    try:  数据 ← JSON 解析(读取文件)
    except JSON 解析失败:
        备份原文件为 “<slot>.json.bak”
        抛出 SaveError(“存档已损坏，已备份为 .bak”)
    if 数据.version > 当前版本:  抛出 SaveError(“版本过新”)
    while 数据.version < 当前版本:
        数据 ← MIGRATIONS[数据.version](数据)
        数据.version += 1
    恢复随机源 → 重建玩家 → 地图（含房间标记）→ 怪物 → 道具 → 进度与日志
```

存档设计有三个刻意为之的选择。**其一，只存 JSON、不用 pickle**：pickle 在反序列化时会执行任意代码，存档一旦被替换就相当于给电脑开后门；而且 pickle 记录的是“模块路径 + 类名”，重命名一次类所有老存档立刻报废。**其二，同时保存种子与状态快照**：只存种子只能“重放整局”，无法回到“当前这一刻”，因为 Mersenne Twister 已经抽取过几百次；额外保存 `getstate()` 的 625 个整数，换来的是读档后怪物每一次 AI 决策都与存档时完全一致。**其三，原子写入**：先写临时文件、`fsync` 强制落盘、再 `os.replace` 原子替换，即使写到一半断电，旧存档也不会被写坏。

实验结果与界面展示

1. 开始界面与设置

程序启动后先显示开始界面（图 4），提供“开始新游戏 / 读取存档 / 设置 / 退出”四项。开始界面期间菜单栏会被隐藏、游戏按键被屏蔽——否则玩家可以一边看开始菜单一边按方向键，后面那局游戏会跟着偷偷走几步。



图 4　开始界面

设置面板（图 5）不是一项一项手写的，而是**由 schema 生成**：`ui/settings.py` 里每增加一条 `SettingSpec`，面板就自动多一行、自动校验、自动持久化到 `data/settings.json`。当前支持全屏模式、自适应难度与难度预设，以后要加“音量”“语言”之类的选项，只需追加一行配置，不必改动界面代码。



图 5　设置面板（schema 驱动）

全屏除了在设置里开关，也可以随时按 `F11` 切换，选择会被记住；退出全屏时窗口尺寸自动还原。配置文件损坏或字段类型不对时一律回退默认值——不能因为一个坏配置把游戏挡在启动界面之外。

2. 主界面与探索

进入游戏后的主界面见图 6：窗口 1300 × 820，左侧是 40 × 25 的地图画布（960 × 600），右侧是角色状态面板（生命/经验条、属性、装备、背包），底部是彩色日志区与状态栏。装备名后会显示强化等级（如“铁剑 +1”），经验条改用白字以保证在蓝条上可读。

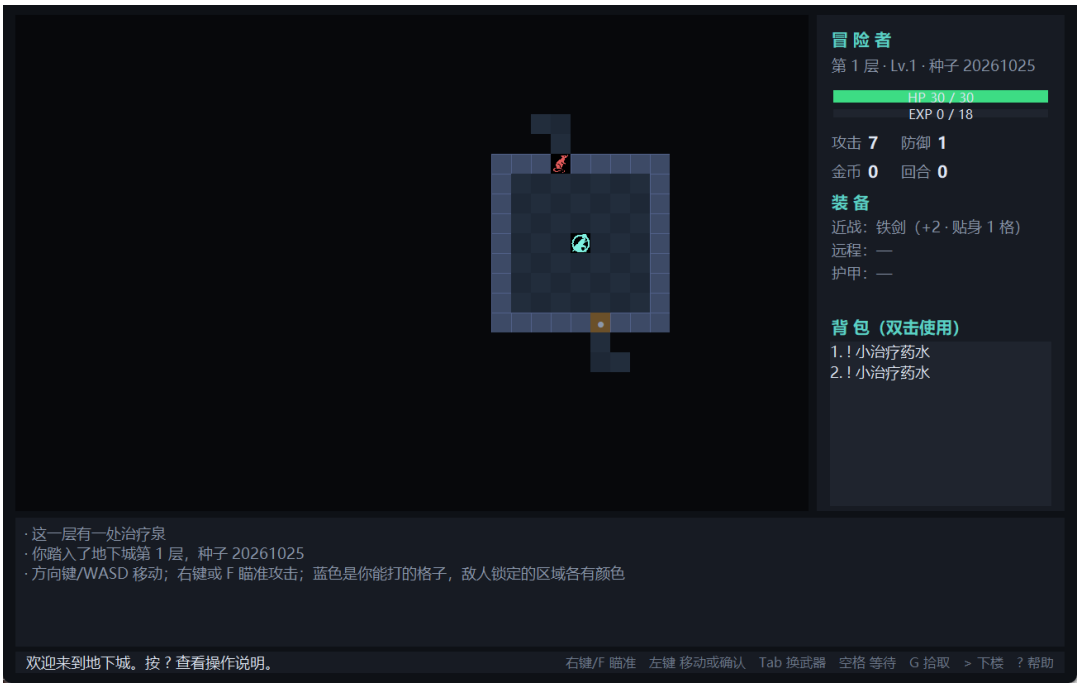


图 6　开局主界面

经过一段探索后（图 7），已探索但当前不在视野内的区域整体压暗到 60%，形成典型的“战争迷雾”效果；日志区用不同颜色区分信息类型——灰色是系统提示、红色是战斗、青色是升级与治疗、金色是道具。

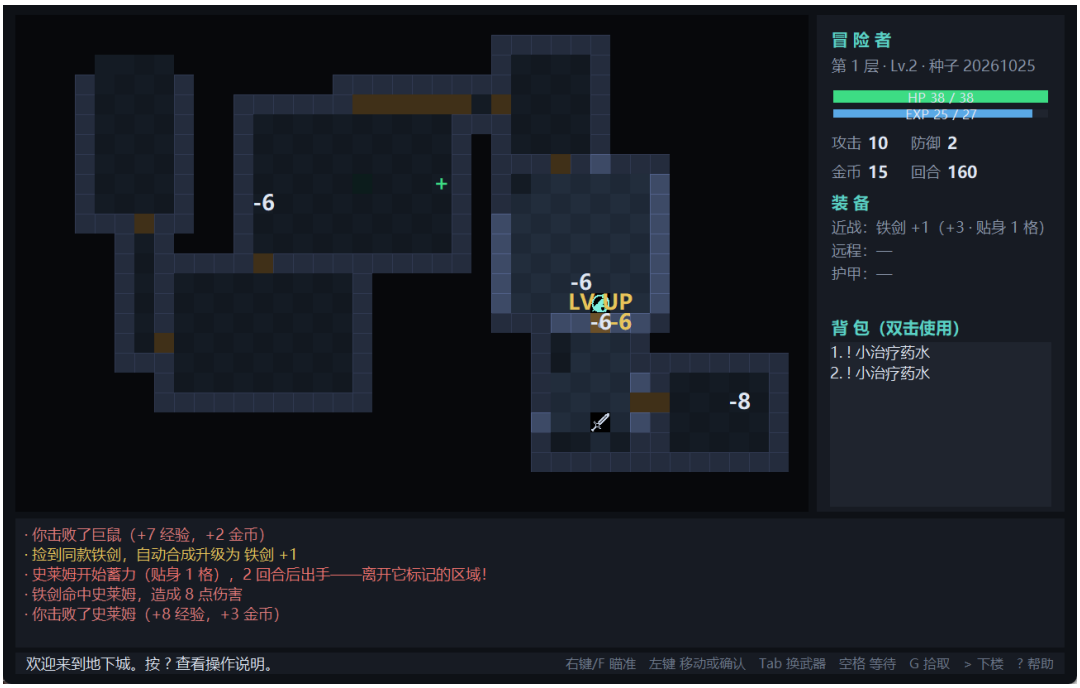


图 7　探索与战争迷雾

3. 策略战斗

图 8 是一次多敌人同时蓄力的战斗：每只敌人的锁定区域颜色不同，范围沿外轮廓描了同色实线，敌人格子右上角的倒计时数字边框与它那片区域同色，数字本身固定白色以保证可读。图中央金色描边的是精英，左侧青绿描边的是治疗兵，地图上的 `$` 与 `≈` 分别是宝库与治疗泉。

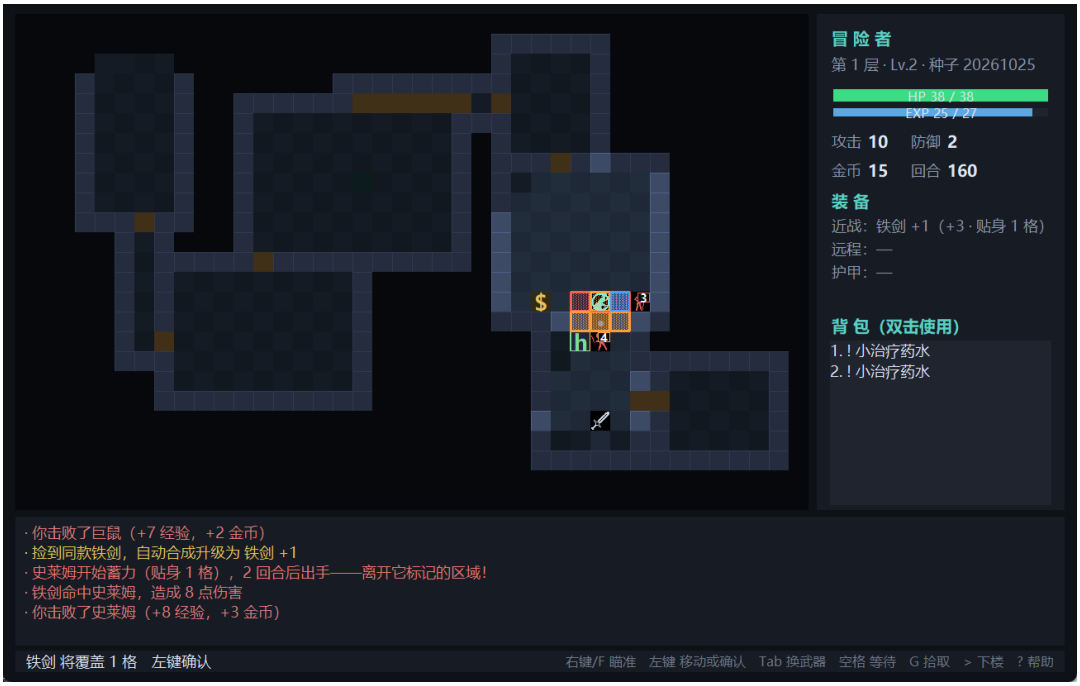


图 8　策略战斗：逐敌配色、精英与治疗兵

这套配色不是装饰。早期所有威胁区都用同一种红色，两三只敌人一起蓄力时就糊成一片，玩家无法判断“这片范围是谁的、还剩几回合”。改成逐敌配色后，颜色本身成了“敌人 → 范围 → 倒计时”三者之间的索引。

4. 弹窗与交互

操作说明窗口（图 9）用双列对照表列出全部快捷键，并在底部补充战场色块图例、精英与治疗兵的识别方式、四类房间的字符。单列放不下 17 条快捷键，会被挤出窗口，因此改为双列布局。



图 9　操作说明

背包窗口（图 10）采用左右分栏：左侧是道具列表，右侧显示选中道具的类型与效果说明；双击或按回车即可使用。装备类道具被拒绝时（例如想用更弱的武器替换现有武器），提示会明确写出“保留现有武器”的理由，并且道具留在背包里。



图 10　背包窗口

一局结束时弹出结算窗口（图 11），显示评级（S/A/B/C/D）、到达层数、等级、回合数、击杀数、金币、输出与承受伤害、伤害交换比、用时与死因。



图 11　结算窗口

排行榜（图 12）按“到达层数 → 击杀数”排序，展示最多 30 条历史记录，顶部汇总总局数、通关率、平均层数与最常见死因。



图 12　历史排行榜

5. 数据可视化

每局结束时，程序自动把 14 项指标写入 `data/records.json`。菜单“查看 → 统计图表”会读取这份数据，用 pandas 聚合后交由 matplotlib 渲染，可左右翻页查看 5 张图（图 13）。

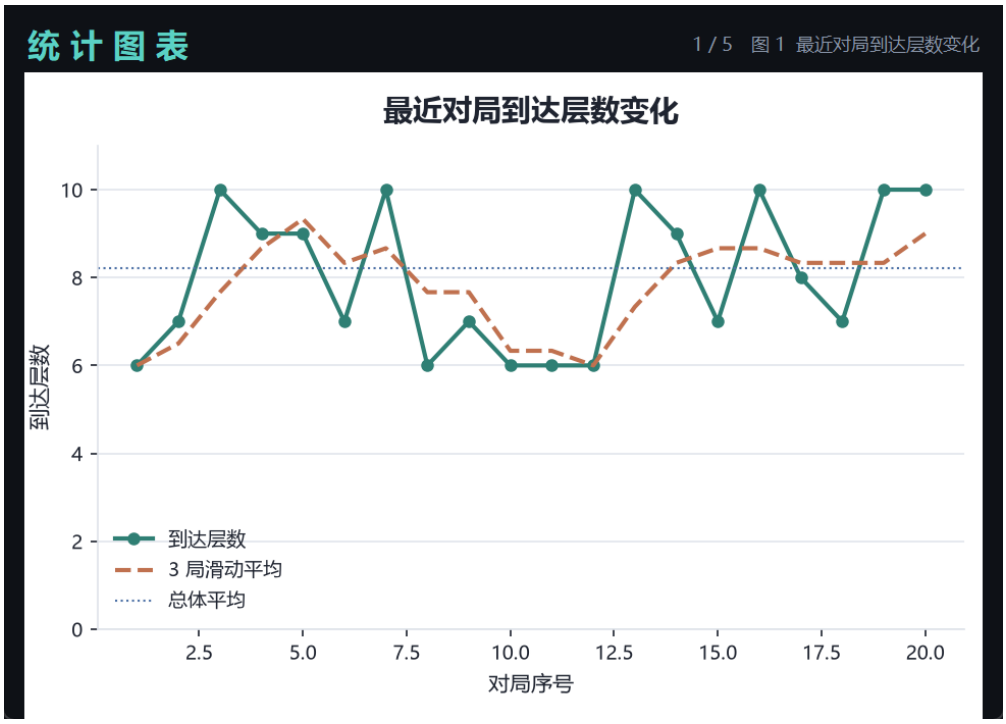


图 13 统计图表窗口

为得到有统计意义的数据，用批量模拟器跑了 40 局完整自动对局（种子 5000~5039）。汇总结果见表 10。

表 10 批量自动对局统计

指标	数值
总对局数	40 局
通关 / 阵亡 / 超时	16 / 22 / 2 局
通关率	40.0%
平均到达层数	8.40 层
平均回合数	1892
平均击杀数	104.3
平均等级	9.5
平均造成 / 承受伤害	3955 / 239

五张图表从不同角度解读这批数据。

图 14 是到达层数的变化曲线。 折线是每局到达的最深层数，虚线是滑动平均，点线是总体平均（8.40 层）。不少对局能推进到第 8 层以上，少数在第 6~7 层折戟，说明中期确实存在压力，但不会把人卡死。

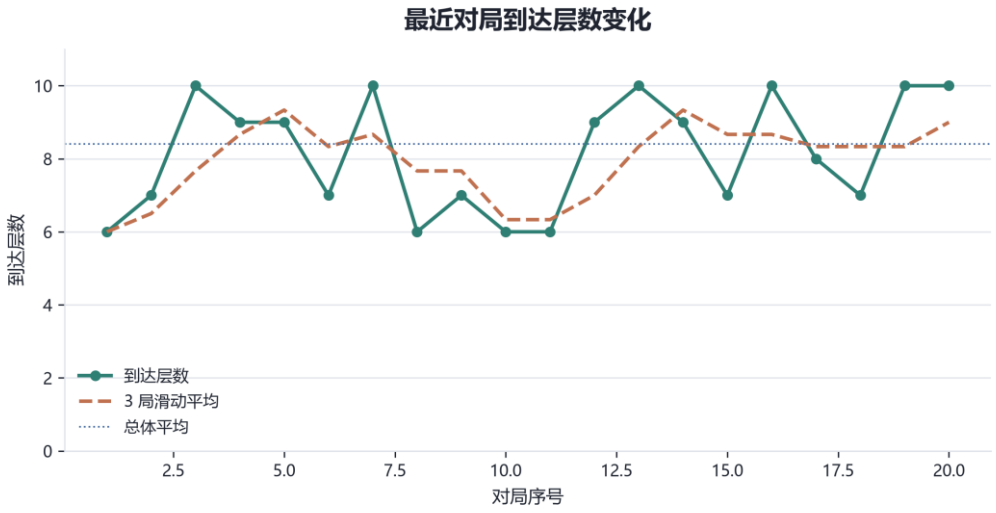


图 14　到达层数趋势

图 15 是阵亡层数分布与死因统计。 左图显示阵亡集中在 第 6 层 7 局、第 7 层 7 局、第 8 层 3 局、第 9 层 5 局；右图显示死因以 大刀兵 8 次、长矛兵 3 次、精英·弓兵 2 次 为主。大刀兵是高伤害、蓄力 4 回合的横扫单位，在狭窄走廊里很难躲开它的三格范围，因此成了头号杀手——这也说明“伤害越高蓄力越长”的设计并没有让高伤害单位变得无害。

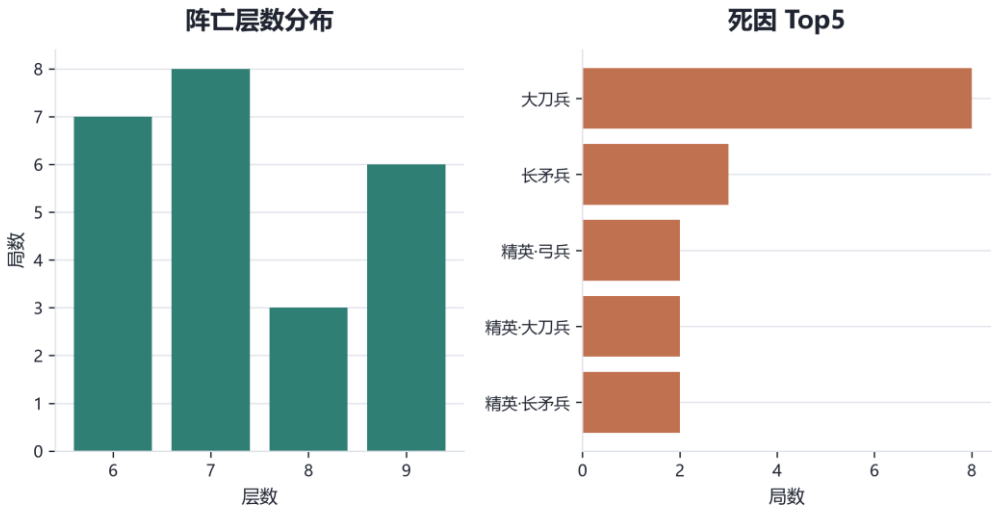


图 15　阵亡层数分布与死因统计

图 16 是回合数与击杀数的散点图。 每个点代表一局，按结果着色，虚线是线性拟合。两者呈明显的正相关，符合“打得越久、杀得越多”的直觉；偏离拟合线的点就是典型的“暴毙局”——回合数不高但击杀数很高，说明一路硬冲、没有补给。

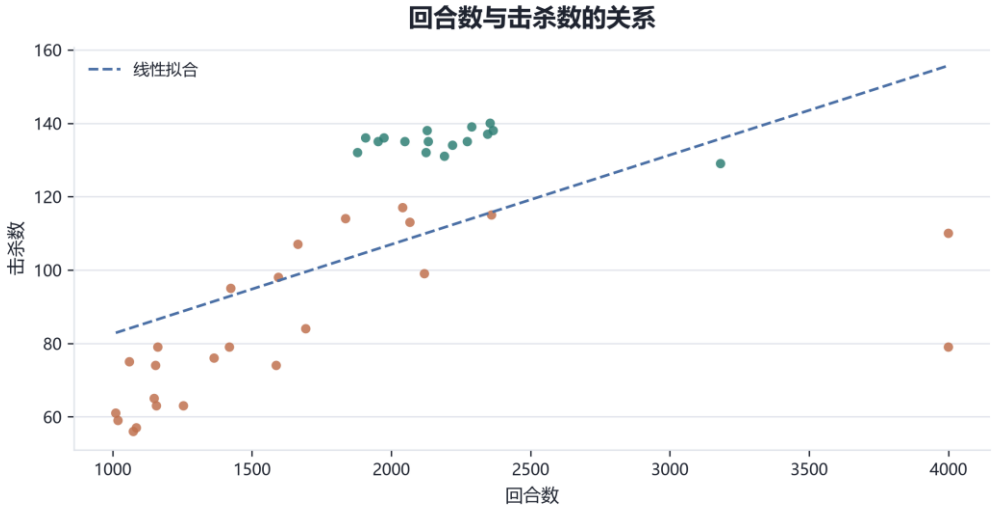


图 16　回合数与击杀数的相关性

图 17 是各层平均停留回合。各层分别为 第 1 层 159 回合、第 2 层 187 回合、第 3 层 231 回合、第 4 层 201 回合、第 5 层 238 回合、第 6 层 225 回合、第 7 层 338 回合、第 8 层 249 回合、第 9 层 314 回合、第 10 层 93 回合。可以看到第 7、9 层明显高于相邻层——这两层正是剖面里的尖峰（20 只

怪 +2 个精英），而第 10 层骤降，因为这一层没有下行楼梯，玩家只需找到并击败 BOSS，不需要地毯式探索。这正是设计意图：最终层应该是“决战场”而不是“探索场”。

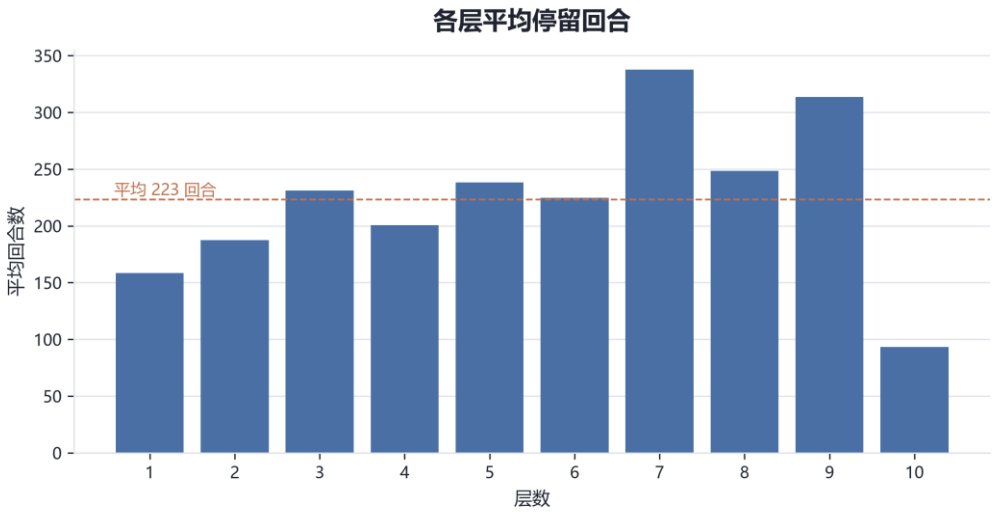


图 17　各层平均停留回合

图 18 是对局结果构成。 环形图显示 40% 通关、其余阵亡或超时。作为一个需要主动探索的 Roguelike，四成左右的通关率意味着“认真玩能赢、乱冲会死”，难度曲线基本落在预期区间。

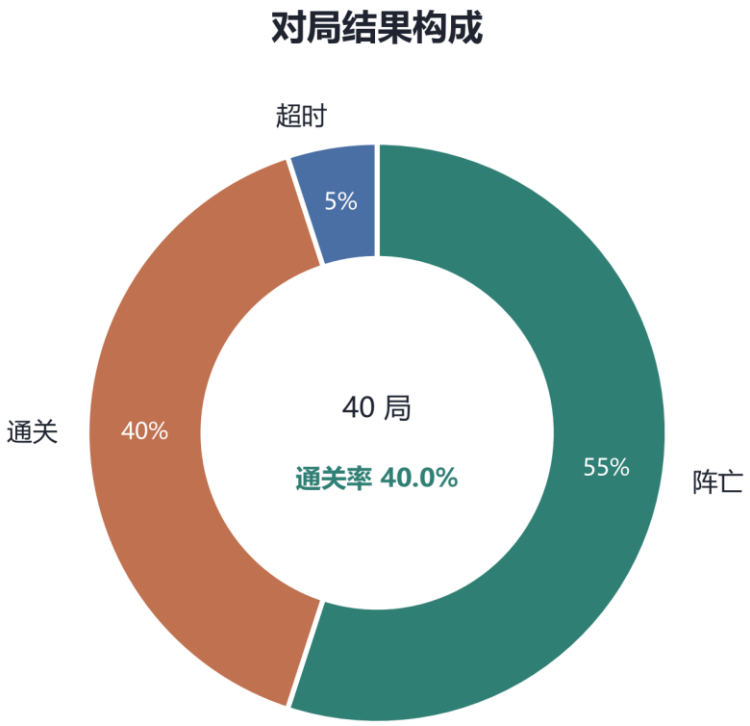


图 18　对局结果构成

6. 数值平衡的迭代过程

这套数值不是一次调好的，而是靠模拟器反复验证出来的。过程中有几个值得记录的转折点。

第一次：怪物成长太慢。 初版怪物攻击每层只增长 10%，而玩家防御每级 +1。模拟 30 局后发现所有对局都能轻松走到第 10 层，图表上是一条毫无起伏的直线——原因是玩家到 10 级时防御已达 13 点，而第 9 层骷髅兵的攻击只有 13 点，伤害被完全抵消。把攻击成长提高到每层 22% 后，阵亡层数才分散到第 6~10 层。

第二次：模拟器自己有问题。 发现自动玩家很少更换装备，导致中期战斗力严重不足。检查后确认这是模拟器的缺陷（真实玩家一定会换装备），补上“自动装备更优武器护甲”的逻辑后，平均到达层数从 8.5 提升到 9.5。

第三次：BOSS 战不可胜。 最初 BOSS 有 420 点生命、32 点攻击，模拟结果显示到达第 10 层的玩家全部阵亡。单独模拟一场“满血玩家 vs BOSS”的决

斗，玩家在第 11 回合倒下时 BOSS 还剩 38 点血——差距不大但始终差一口气。最终做了两处调整：BOSS 降到 340 点生命、23 点攻击；进入第 10 层时让玩家回满生命（剧情上解释为“在通往深渊的石阶上稍作休整”）。后者解决的是更根本的问题——原设计下玩家往往带着残血和空药水进场，BOSS 战变成了比谁先掉血的算术题。

第四次：加入精英、治疗兵与房间之后重新配平。 因为整体强度上升，把怪物攻击成长从 0.30 调到 0.37，把治疗泉的回复量从 35% 降到 28%。这一步对参数极其敏感：0.36 时通关率 58%，0.38 掉到 37.5%，0.40 直接 25%，最终定在 0.37，通关率稳定在四成左右——与改造前的 47.5% 基本持平，说明新增内容后曲线被重新配平，而不是简单地把游戏变难。

第五次：模拟器自身的两次死循环。 一次是机器人“敌人一起手就躲、躲完敌人又起手”，整局打不出输出；一次是它坚持“清完一层所有怪才下楼”，在大层上耗尽回合上限。前者通过“连续躲 3 次后强制换血或脱离”解决，后者给猎杀加了楼层预算。修完之后，40 局里因步数上限而未完成的对局从 4 局降到 2 局。

7. 界面性能优化

如果每回合都 `delete("all")` 重画整张地图，1000 个格子会造成明显的输入延迟。本项目的做法是**启动时一次性创建全部图元并缓存 item id**，之后每回合只对“外观签名”发生变化的格子调用 `itemconfig`。实测一次移动通常只有 2 个格子（起点与终点）加上视野边缘的十几格需要更新，单帧更新量不到 50 次，从按键到画面刷新几乎无感。迷雾的三种状态通过改同一个矩形的填充色实现，不额外增加图层，也不会出现遮挡错位。

此外还有三处细节：伤害飘字数量硬性限制为 20 个，超出时丢弃最旧的；飘字动画用 40 毫秒节流而不是每帧重绘；鼠标悬停只移动一个高亮矩形，不做任何重计算。

8. 可替换的图标资源

游戏内所有物件的图标都支持换成自己的图片，不需要改代码。资源清单位于 `assets/icons.json`，每条只声明三件事：键名、中文名、图片相对路径。查

找采用“三级回退”：某件道具先找 `item.<道具名>`，没有就用 `item.<类别>`（所有药水共用一张），再没有就用 `item`，最后回退到字符绘制。替换后按 `F12` 即可热重载，不必重启程序。

实现上有三个细节：`PhotoImage` 必须由图标集对象长期持有，否则 Python 对象被回收后画布上的格子会凭空消失；生成迷雾变暗版时必须把 RGBA 拆开、只压 RGB 通道，否则半透明图标的边缘会糊成黑边；热重载要按“先建新、再刷新、最后丢旧”的顺序，反过来会有一瞬间画布仍指着已释放的图像。墙和地板**故意不做成图片**：它们承担着战争迷雾的三态显示，现在只改一个填充色就能完成，换成贴图收益几乎为零。

小结。 这一章的证据可以归为三类：界面截图证明功能确实跑得起来；40 局批量对局与 5 张图表证明难度曲线落在预期区间；迭代记录则说明这些数值不是一次写对的，而是被数据反复推翻后收敛出来的。三者叠加，才能回答“这个游戏到底能不能玩”这个问题——单靠几张截图是回答不了的。

测试与异常处理

1. 测试方法

测试使用 Python 标准库自带的 `unittest`，共 11 个测试文件、**153 个测试用例**。选择 `unittest` 而不是 pytest 的理由是不增加第三方依赖，同时 pytest 也能直接运行 unittest 用例，两种方式都兼容。

由于逻辑层完全不依赖界面库，全部用例可以在无显示器环境下运行，实测 153 个用例耗时约 0.35 秒（图 19）。这一点很重要：它意味着每次改完数值都可以立刻把全部规则回归一遍，而不必打开窗口手动点。

```
$ python -m unittest discover -s tests
...................................................
...................................................
...................................................

----------------------------------
Ran 153 tests in 0.347s

OK
```

图 19　单元测试运行结果（153 个用例全部通过）

2. 测试覆盖的重点

表 11　测试文件与用例分布

测试文件	用例数	覆盖内容
test_dungeon.py	8	房间不重叠（100 个种子）、地图全连通、楼梯不贴脸、视野遮挡、序列化往返
test_combat.py	10	伤害下限、暴击倍率、落空、负数参数拒绝、扣血钳制、治疗封顶、连续升级
test_save.py	10	存读往返、随机数复现、坏 JSON、缺字段、版本过新、非法槽位、战绩追加
test_boundary.py	18	撞墙不消耗回合、非法方向、越界查询、空背包、非法索引、满血喝药、死亡与超时判定

测试文件	用例数	覆盖内容
test_tactics.py	24	四种攻击形状、墙体阻断、蓄力单调性、蓄力落下、躲避预告、存档兼容
test_craft.py	10	同名装备合成、满级上限、背包满仍可合成、伪造等级截断、旧档兼容
test_equipment.py	11	装备槽映射、只许更强替换、同级拒绝、护甲降级拒绝、武器阶梯单调
test_progression.py	29	难度锯齿结构、精英、治疗兵、四类房间、自适应、难度预设、新字段往返
test_settings.py	14	schema 完整性、持久化、坏配置回退、未知键保留、扩展通道
test_icons.py	14	清单缺失/坏 JSON/版本不符、非法条目过滤、三级回退顺序、tint 校验
test_architecture.py	5	逻辑层无界面依赖、界面层无业务规则、图标与设置模块可无头运行

其中“房间不重叠”与“地图全连通”两个用例都对 **100 个随机种子**逐一验证，因为地图生成是随机算法，只测一个种子等于没测。同样地，难度剖面不是靠肉眼看表，而是让测试去检查“存在回落”与“尖峰后有恢复房间”——把设计意图写成断言。

3. 三组重点测试记录

第一组：正常流程（完整一局 + 存读档一致性）

操作步骤：启动游戏 → 种子 20261025 → 移动探索 → 瞄准攻击击杀哥布林 → 获得经验并升级 →踩到药水自动拾取 → 下到第 2 层 → 按 F5 存档 → 关闭程序 → 重新启动 → 按 F9 读档。

表 12　存档前后一致性核对

检查项	存档前	读档后
玩家坐标	(10, 4)	(10, 4)
生命值	30 / 30	30 / 30
背包道具	小治疗药水 ×2	小治疗药水 ×2
怪物数量与位置	4 只	4 只，坐标完全相同

检查项	存档前	读档后
地面道具	3 件	3 件，坐标相同
趣味房间标记	治疗泉 1 处	1 处，坐标相同
楼梯坐标	(37, 1)	(37, 1)
地图地形（25 行字符串）	完全一致	完全一致
读档后连续 10 个随机数	890565297 …	完全相同

第二组：非法输入（人为破坏存档与配置）

表 13　非法输入的处理结果

破坏方式	预期行为	实际结果
把 save1.json 改成不合法 JSON	提示损坏并备份	弹出“存档已损坏，已备份为 .bak”，原文件保留
删除 JSON 中的 player 字段	提示内容不完整	弹出“存档内容不完整”，程序不退出
把 version 改成 99	提示版本不兼容	弹出“存档版本高于当前程序，请更新程序”
把道具 key 改成不存在的值	拒绝加载	抛出“存档中存在未知道具”，被包装成 SaveError
settings.json 里把 fullscreen 写成字符串	回退默认值	按默认值启动，不报错、不崩溃
输入负数种子 / 非数字种子	拒绝并提示	状态栏提示“种子必须是正整数”

第三组：边界情况

表 14　边界情况测试

场景	操作	结果
撞墙	在封闭房间里向墙移动	返回 False，回合数与坐标均不变，提示“前面是石墙”
越界查询	查询 (-1, 0)、(999, 999)	一律视为不可通行，不抛异常
满血喝药	生命 30/30 时使用治疗药水	提示“生命已满”，道具不被消耗
空背包使用道具	清空背包后按数字键 1	提示“这个格子里没有道具”，不抛 IndexError

场景	操作	结果
背包已满	容量 24 时继续拾取	提示“背包已满”，道具留在地面
用弱武器替换强武器	装备符文长剑后使用铁剑	拒绝并提示“保留现有武器”，道具仍在背包
祭坛前残血	生命低于 30% 时踩上祭坛	拒绝交易，且不消耗这次机会（养好伤可再来）
击破 BOSS	在第 10 层击败深渊领主	状态切为 win，结算显示“通关成功”

4. 异常处理策略

项目在四个层面做了分级容错，原则是“能在下层安全返回的就不要抛异常，必须抛的要在上层转成用户看得懂的中文提示”。

表 15　分层容错策略

层面	策略
底层逻辑	非法数值抛 ValueError（如负数伤害）；越界坐标一律当作墙；道具使用失败返回 (False, 原因) 而不是抛异常
存档层	所有读写异常统一包装成 SaveError，消息用中文描述得能直接给用户看；坏文件先备份再报错
界面层	捕获 SaveError 并转成弹窗；任何写操作失败都不会让窗口失去响应
入口层	main.py 捕获顶层异常并打印完整堆栈，同时在控制台暂停，避免双击运行时窗口一闪而过

实验总结

1. 完成情况

本次大作业完成了一个 6913 行核心代码（另加 3196 行工具脚本与 1992 行测试）的完整 Python 项目，实现了程序化地图生成、回合制策略战斗、道具与装备、多层探索、精英与治疗兵、四类趣味房间、锯齿难度曲线、BOSS 战、JSON 存档、数据统计、开始界面与设置、图形界面等全部预定功能。153 个单元测试全部通过，40 局批量模拟验证了难度曲线的合理性，并生成了 3 张流程图、5 张统计图表与 10 张界面截图作为实验证据。

2. 主要收获

第一，真正理解了“分层”的价值。 刚开始写的时候，我习惯性地在界面回调里直接写“如果这个格子有怪物就扣血”。直到要写单元测试时才发现，这样写根本没法测——每跑一次测试就要弹一个窗口出来。改成“界面只调用意图方法、逻辑层推送事件”之后，测试变成了一件轻松的事，153 个用例 0.35 秒跑完，而且能在没有显示器的环境里跑。这个转变是本次作业最大的收获。

第二，学会用数据代替直觉。 调整数值时我的直觉是“BOSS 有 420 点血应该挺有压迫感”，但模拟器告诉我玩家根本打不过；把血降到 340 之后通关率也只有 3.3%，直到单独模拟一场决斗、逐回合看血量变化，才定位到真正的问题是“玩家带着残血进场”而不是“BOSS 太强”。如果没有模拟器，这个问题大概只能靠反复手动试玩来发现。

第三，同一个概念只允许一个映射函数。 本次修过一个很隐蔽的缺陷：近战武器换上后右侧面板不更新、伤害也不变。根因是装备方法把武器的**分类**（`melee`）当成了装备字典的**键**，于是剑被写进 `equipment["melee"]` 这个凭空出现的键，而面板与战斗都读 `equipment["weapon"]`；更麻烦的是“已经装备着”的判重也一起失效了。修法不是到处打补丁，而是给武器加一个 `equip_slot` 属性作为唯一出口，所有调用点统一走它。这件事让我明白：**重复的映射逻辑迟早会分叉**。

第四，踩了不少工程细节的坑。 比如高分屏下 tkinter 默认不是 DPI 感知的，界面会被系统拉伸变糊；比如 `random` 的全局状态会被任何第三方库污染，所以项目里所有随机都必须走同一个 `Rng` 实例；比如 `PhotoImage` 被垃圾回收后画布上的图会凭空消失；比如用 `create_line` 画多段不连通的边界时，把坐标串一次传进去会把它们连成横穿区域的斜线。这些都不是课本上会讲的，但确实是“能用的程序”和“好用的程序”之间的差距。

3. 存在的不足与改进方向

1. **怪物 AI 比较简单。** 目前只有追击 / 游荡 / 守卫 / 支援四种模式，追击采用贪心走位，遇到墙角容易卡住。后续可以加入 A* 寻路，或让怪物学会“绕后”“集火”。

2. **地图类型单一。** 只有房间加走廊一种生成方式，缺少洞穴、水域等要素，可以引入 BSP 或元胞自动机生成洞穴地形。

3. **没有音效与动画。** 目前只有伤害飘字，怪物移动是瞬移。可以用 pygame.mixer 加音效，或给移动加插值动画。

4. **自适应难度只用了血量与击杀进度两个信号。** 更细致的做法是记录玩家的“有效输出 / 承受伤害比”“药水消耗速度”，但那也更难解释给玩家听，需要在“聪明”和“可理解”之间再权衡。

5. **统计维度还可以更丰富。** 现在只统计每局的汇总数据，没有记录单场战斗的细节，因此无法分析“哪种怪物造成的伤害最多”这类问题。

4. 结语

从一张空白的文件夹到一个能跑、能存、能看数据的完整游戏，这次大作业让我第一次完整体验了软件开发的流程：需求分析 → 设计 → 编码 → 测试 → 调优 → 文档。代码量不算大，但每一个模块都要想清楚“它该负责什么、不该负责什么”，这种边界意识可能比多写几百行代码更有价值。

参考文献

[1] Csikszentmihalyi M. Flow: The Psychology of Optimal Experience[M]. New York: Harper & Row, 1990.

[2] Schell J. The Art of Game Design: A Book of Lenses[M]. 3rd ed. Boca Raton: CRC Press, 2019.

[3] Hunicke R. The Case for Dynamic Difficulty Adjustment in Games[C]// Proceedings of the 2005 ACM SIGCHI International Conference on Advances in Computer Entertainment Technology (ACE '05). Valencia: ACM, 2005: 429-433. DOI: 10.1145/1178477.1178573.

[4] Adams E. The Designer's Notebook: Difficulty Modes and Dynamic Difficulty Adjustment[J/OL]. Game Developer, 2008.

[5] Shaker N, Togelius J, Nelson M J. Procedural Content Generation in Games[M]. Cham: Springer, 2016.

[6] Python Software Foundation. Python 3 Documentation[EB/OL]. https://docs.python.org/3/

[7] Python Software Foundation. tkinter — Python interface to Tcl/Tk[EB/OL]. https://docs.python.org/3/library/tkinter.html